06

Protegendo o contexto: os arquivos de exclusão

Todo mundo pensa no que o agent precisa ler. Quase ninguém pensa no que ele *nunca* deveria ler. Sem essa proteção, o agent desperdiça a janela de contexto com `node_modules`, pode entregar os segredos do seu `.env` ao modelo e toma decisões baseadas em arquivos de build desatualizados. Este material cobre o mecanismo que resolve isso em cada ferramenta.

A IDEIA CENTRAL DESTE MATERIAL

O arquivo de exclusão é o .gitignore da janela de contexto: mesma sintaxe, mesmo conceito — mas protegendo o que o agent lê, não o que o git versiona. É o artefato mais barato do Harness e um dos primeiros a criar.

1 Os três estragos de não ter exclusão

ESTRAGO	COMO ACONTECE
Desperdício de tokens	Sem configuração, o agent pode consumir mais de 120.000 tokens em projetos *pequenos* lendo `node_modules` e afins — em projetos maiores, 200K+, saturando a janela com código que não é seu.
Vazamento de segredos	Chaves e credenciais do `.env` chegando ao modelo. Regra dura: **nunca deixe segredo real no repositório** durante desenvolvimento com agents.
Decisões erradas	O agent lê builds desatualizados, cache de compilador ou configs de CI que não refletem o estado real — e decide com base neles.

2 O arquivo certo em cada ferramenta

FERRAMENTA	ARQUIVO	OBSERVAÇÕES
Claude Code	`.claudeignore`	Na raiz do projeto. Atenção: há relatos de leitura do `.env` mesmo listado — não confie segredos reais a ele.
OpenCode	`.gitignore`	Ainda sem arquivo próprio (há pedido aberto de `.opencodeignore`); respeita o .gitignore por padrão.
Gemini CLI	`.geminiignore`	Sintaxe do .gitignore. Mudanças exigem **reiniciar a sessão** para ter efeito.
Gemini Code Assist	`.aiexclude`	Versão para a extensão de IDE do Google; mesma sintaxe.
Codex CLI	`.codexignore`	Existe, mas com bugs conhecidos de não ser respeitado — o .gitignore é a proteção mais confiável.
Cursor	`.cursorignore`	Também há `.cursorindexingignore` para excluir da indexação de código.

> **Nota de validade:** esse mapa muda rápido — ferramentas ganham e corrigem suporte a cada versão. Antes de confiar num arquivo de exclusão, teste (seção 5) em vez de assumir.

3 Template base para qualquer projeto

```
# Dependências
node_modules/
.venv/
vendor/
__pycache__/
*.egg-info/

# Build e gerados
dist/
build/
.next/
out/
*.pyc  *.pyo  *.js.map  *.css.map

# Segredos — NUNCA devem chegar ao modelo
.env
.env.*
*.pem  *.key  *.p12
*_rsa  *_dsa
```

```
credentials.json
service-account*.json

# Banco de dados local
*.sqlite  *.sqlite3  *.db

# Uploads e mídia de usuário
/uploads/
/media/
/storage/

# Cache e logs
.cache/
*.log
logs/
tmp/
.parcel-cache/
.eslintcache

# IDE e OS
.vscode/settings.json
.idea/
.DS_Store
Thumbs.db

# Coverage e testes gerados
coverage/
.nyc_output/
htmlcov/
```

4 Proteção em camadas

CAMADA	O QUE FAZ
1 — .gitignore	A base. Protege contra commits acidentais e já cobre OpenCode e Codex CLI.
2 — Arquivo da ferramenta	`.claudeignore`, `.geminiignore`, `.cursorignore` — a proteção específica da janela de contexto.
3 — Segredos fora do repositório	Vault do provedor, secrets do CI/CD (GitHub Actions, Vercel), `.env` apenas local e sempre no .gitignore.

Interoperabilidade

Uma estratégia prática para times com várias ferramentas: configure o `.gitignore` com **todas** as exclusões e use os arquivos específicos com `!` (include) apenas para o que o git ignora mas o agent precisa ler — por exemplo, `!.env.example`.

▸ REGRA DE OURO

Se o arquivo não é **código fonte, teste ou documentação** do projeto: exclua. Na dúvida, exclua — e adicione de volta se o agent precisar.

5 Checklist antes de rodar qualquer agent

- ☐ O `.gitignore` cobre dependências, builds, segredos e cache?
- ☐ O arquivo de exclusão específico da ferramenta existe na raiz?
- ☐ Nenhum segredo real está no repositório (`.env`, keys, certificados)?
- ☐ O agent consegue acessar o que realmente precisa (nada essencial excluído por engano)?
- ☐ Você **testou** que o agent não lista arquivos excluídos ao buscar contexto?

▸ COLA PARA FIXAR

- Arquivo de exclusão = **.gitignore do contexto**: define o que o agent nunca lê.
- Sem ele: **tokens desperdiçados** (120K+ até em projeto pequeno), **segredos expostos** e decisões sobre arquivos gerados.
- Cada ferramenta tem o seu: `.claudeignore`, `.geminiignore`, `.cursorignore` — OpenCode usa o próprio .gitignore.
- Proteja em **camadas**: .gitignore + arquivo da ferramenta + segredos fora do repositório.
- Nenhum ignore é garantia para segredos: **segredo real não fica no repositório**, ponto.
- Regra de ouro: não é código, teste ou doc? **Exclua.**